


```
# mi primera cuaderno  
# uso del comando print(.)  
print("hola mundo")
```

hola mundo

```
# Mi primer programa en Python  
print("¡Hola. mundo desde Google Colab!")
```

```
# mi primera cuaderno
# uso del comando print(.)
print("hola mundo")
```

hola mundo

```
print("mi nombre es:")# brahian
```

mi nombre es:

```
# Mi primer programa en Python
print("¡Hola, mundo desde Google Colab!")
print("Python es increíble")
print("¡Estoy aprendiendo a programar!")
```

¡Hola, mundo desde Google Colab!
Python es increíble
¡Estoy aprendiendo a programar!

```
# Personaliza tu información
nombre = "brahian "
edad = 22
ciudad = "medellin"
lenguaje_favorito = "Python"
```

```
print(f"Hola, soy {nombre}")
print(f"Tengo {edad} años")
```

Hola, soy brahian
Tengo 22 años

```
print(f"hola, soy {nombre}. tengo{edad} años")
```

hola, soy brahian . tengo22 años

```
print(f"vivo en la ciudad de {ciudad} y programo en {lenguaje_favorito}")
```

vivo en la ciudad de medellin y programo en Python

```
texto = "hola mundo python"

texto_mayusculas = texto.upper()
longitud_texto = len(texto)

print(f"Mayúsculas: {texto_mayusculas}")
print(f"Longitud: {longitud_texto}")
```

```
#Ejercicio 3: Python como calculadora
#Explora las operaciones matemáticas básicas:
```

```
# Python como calculadora
# Operacion de suma
print("=== Operaciones Básicas ===")
print(f"2 + -3 = {2 + -3}")
```

```
=== Operaciones Básicas ===
2 + -3 = -1
```

```
# Python como calculadora
# Operacion de resta
print("=== Operaciones Básicas ===")
print(f"10 - 4 = {10 - 4}")
```

```
=== Operaciones Básicas ===
10 - 4 = 6
```

```
# Python como calculadora
# Operacion de multiplicacion
print("=== Operaciones Básicas ===")
print(f"6 * 7 = {6 * 7}")
```

```
=== Operaciones Básicas ===
6 * 7 = 42
```

```
# Python como calculadora
# Operacion de multiplicacion
print("=== Operaciones Básicas ===")
print(f"-4 * 5 = {-4 * 5}")
```

```
=== Operaciones Básicas ===
-4 * 5 = -20
```

```
# Python como calculadora
# Operacion de division
print("=== Operaciones Básicas ===")
print(f"15 / 3 = {15 / 3}")
print(f"15 / 2 = {15 / 2}")
print(f"1 / 3 = {1 / 3}")
```

```
=== Operaciones Básicas ===
15 / 3 = 5.0
15 / 2 = 7.5
1 / 3 = 0.3333333333333333
```

```
# Python como calculadora
# Operacion de potenciacion
print("=== Operaciones Básicas ===")
print(f"2 ** 8 = {2 ** 8}")
print(f"2 ** 3 = {2 ** 3}")
print(f"2 ** 4 = {2 ** 4}")
print(f"2 ** 5 = {2 ** 5}")
```

```
=== Operaciones Básicas ===
2 ** 8 = 256
2 ** 3 = 8
2 ** 4 = 16
2 ** 5 = 32
```

```
# Python como calculadora
# Operacion de residuo de division
print("=== Operaciones Básicas ===")
print(f"17 % 5 = {17 % 5}")
```

```
=== Operaciones Básicas ===
17 % 5 = 2
```

```
# Python como calculadora
# prueba para el residuo de division
print("=== Operaciones Básicas ===")
print(f"3 * 5 + 17 % 3 = {3 * 5 + 17 % 3}")
```

```
=== Operaciones Básicas ===
3 * 5 + 17 % 3 = 17
```

```
# Ejercicio 4: Prueba en Visual Studio Code Online
#También puedes probar estos ejercicios en Visual Studio Code Online:
```

```
# Crear un archivo Python en VS Code
# 1. Haz clic en "Abrir Visual Studio Code" arriba
# 2. Crea un nuevo archivo: archivo.py
# 3. Copia cualquiera de los ejercicios anteriores
# 4. Guarda el archivo y ejecútalo
```

```
print("¡Hola desde Visual Studio Code!")
nombre = input("¿Cuál es tu nombre? ")
print(f"¡Bienvenido a Python, {nombre}!")
```

```
¡Hola desde Visual Studio Code!
¿Cuál es tu nombre? Brahian
¡Bienvenido a Python, Brahian !
```

```
#ingresa 2 numeros tal que el a%b=2
a =input("ingrese numero a")
b =input("ingrese numero b")
print(f"a % b = {ingrese numero a % ingrese numero b}")
```

```
File "/tmp/ipykernel\_658/3529166479.py", line 4
    print(f"a % b = {ingrese numero a % ingrese numero b}")
    ^
```

SyntaxError: invalid syntax. Perhaps you forgot a comma?

Pasos siguientes: [Explicar error](#)

¿Qué es GitHub?

Una breve introducción

Tu Nombre

28 de julio de 2026

Resumen

Este documento ofrece una descripción concisa de GitHub, una plataforma fundamental en el desarrollo de software moderno. Se explican sus conceptos clave, su relación con Git y su importancia para la colaboración y el control de versiones.

1. Introducción

En el mundo del desarrollo de software, la colaboración y el seguimiento de cambios son esenciales. GitHub ha emergido como la plataforma líder para alojar, revisar y gestionar proyectos de código, convirtiéndose en una herramienta indispensable para desarrolladores de todo el mundo.

2. ¿Qué es GitHub?

GitHub es una plataforma de desarrollo colaborativo que permite a programadores y equipos trabajar juntos en proyectos. Se basa en el sistema de control de versiones **Git**, y añade una capa social y de gestión que facilita el trabajo en equipo.

2.1. Git vs. GitHub

Es importante distinguir entre **Git** y **GitHub**. Git es un sistema de control de versiones distribuido que rastrea los cambios en los archivos. GitHub es un servicio en la nube que aloja repositorios Git y proporciona una interfaz web, herramientas de colaboración y funcionalidades adicionales.

3. Características principales

- **Repositorios:** Espacios de almacenamiento para proyectos, que pueden ser públicos o privados.
- **Ramas (Branches):** Permiten trabajar en diferentes versiones de un proyecto simultáneamente.
- **Pull Requests:** Solicitudes para fusionar cambios de una rama a otra, facilitando la revisión de código.
- **Issues:** Sistema de seguimiento de tareas, errores y propuestas.
- **GitHub Actions:** Automatización de flujos de trabajo (CI/CD).

4. Importancia en el desarrollo actual

GitHub no solo es un lugar para almacenar código. Es un ecosistema donde se fomenta el código abierto (open source), la revisión por pares y la documentación colaborativa. Hoy en día, es el estándar de facto para la gestión de proyectos de software, utilizado por empresas, comunidades y desarrolladores independientes.

5. Conclusión

GitHub ha revolucionado la forma en que se escribe y comparte software. Su integración con Git, sus po-

tentes herramientas de colaboración y su enfoque en la comunidad lo convierten en una herramienta esencial para cualquier persona o equipo que desarrolle software.

Referencias

- [1] Git - Sitio oficial. <https://git-scm.com/>
- [2] GitHub - Guía de inicio rápido. <https://docs.github.com/es/get-started>

Protocolo de Investigación Experimental

Aprendizaje de Rust: Meta-aprendizaje vs. Asistencia Inteligente Justo a Tiempo

Brahian Marín Giraldo
Biología 2026-2
Universidad CES

30 de julio de 2026

Resumen

El lenguaje de programación Rust impone un estricto modelo de propiedad, préstamo y tiempos de vida que previene errores de memoria, pero dificulta su aprendizaje. Este estudio empírico compara dos enfoques pedagógicos: el aprendizaje fundamentado en teoría (Aprender a Aprender) y el aprendizaje justo a tiempo asistido por IA (JIT). Con una muestra de 100 estudiantes de ingeniería, se miden métricas cuantitativas y cualitativas para evaluar retención, depuración y comprensión profunda. Los resultados esperados matizarán las afirmaciones de Raj Reddy sobre el rol de los tutores virtuales en la educación universitaria.

1. Introducción y Marco Teórico

Rust es un lenguaje de programación que garantiza la seguridad de la memoria mediante un sistema de *ownership*, *borrowing* y *lifetimes*. Aunque estas características lo hacen robusto, su curva de aprendizaje es pronunciada; muchos programadores luchan con el compilador `rustc`. La llegada de asistentes de IA como ChatGPT y Copilot ofrece una vía para superar estas barreras, proporcionando sugerencias y explicaciones bajo demanda.

La teoría de **Raj Reddy** sobre la *autodidaxia asistida por IA* sostiene que, en una era de abundancia tecnológica, los estudiantes pueden aprender de manera más eficiente si cuentan con tutores virtuales que los guíen en la resolución de problemas concretos. Reddy argumenta que la IA puede personalizar el aprendizaje y fomentar la curiosidad, pero advierte sobre el riesgo de una comprensión superficial si no se desarrollan habilidades metacognitivas.

Este estudio compara dos métodos pedagógicos para la enseñanza de Rust:

- **Grupo Control – Aprender a Aprender (AaA):** Recibe instrucción en teoría de tipos, semántica de lenguajes, modelos de memoria (heap/stack) y diseño conceptual, antes de codificar.

- **Grupo Experimental – Aprender Justo a Tiempo (JIT):** Se enfrenta directamente a la construcción de una aplicación, utilizando asistentes de IA para resolver errores del compilador conforme aparecen.

El objetivo es medir la efectividad de cada método en términos de retención, capacidad de depuración y comprensión profunda de los conceptos de memoria.

2. Metodología del Experimento

2.1. Muestreo y Participantes

- **Población:** Estudiantes de Ingeniería de Sistemas de universidades colombianas.
- **Muestra:** 100 estudiantes seleccionados mediante muestreo estratificado por nivel de conocimiento previo en programación (bajo, medio, alto). Se asignan aleatoriamente a dos grupos de 50.
- **Criterios de inclusión:** Haber cursado al menos un curso de programación estructurada; sin experiencia previa en Rust.
- **Duración:** 6 semanas, con sesiones de 4 horas semanales (24 horas totales).

2.2. Diseño Experimental

- **Variable independiente:** Método pedagógico (AaA vs. JIT).
- **Variables dependientes:**
 - Rendimiento en pruebas de código (corrección, eficiencia).
 - Tiempo de resolución de errores de borrowing.
 - Capacidad de refactorización sin ayuda.
 - Comprensión teórica (examen conceptual).
 - Número de invocaciones a la IA (solo grupo JIT).

2.3. Procedimiento

Semana 1–2 (Fase de instrucción):

- **Grupo AaA:** Clases magistrales sobre teoría de tipos, memoria (stack/heap), ownership, borrowing y lifetimes. Ejercicios guiados sin IA.
- **Grupo JIT:** Se les presenta un proyecto (ej. un analizador de texto) y se les anima a usar ChatGPT/Copilot para resolver los errores del compilador. No reciben teoría previa.

Semana 3–4 (Práctica supervisada): Ambos grupos trabajan en proyectos similares (pero no idénticos) con complejidad creciente. El grupo AaA no usa IA; el JIT puede usarla libremente. Se registran métricas (tiempo, invocaciones, errores).

Semana 5 (Evaluación formativa): Prueba práctica de depuración: se entrega un código con errores de ownership y lifetimes; deben corregirlo sin IA (ambos grupos). Se mide tiempo y precisión.

Semana 6 (Evaluación sumativa):

- Examen teórico (preguntas de opción múltiple y abiertas) sobre conceptos de memoria y borrowing.
- Proyecto final: construir una aplicación de consola en Rust sin asistencia (en laboratorio controlado). Se evalúa funcionalidad, estilo y manejo de memoria.

3. Plan de Recolección y Análisis Estadístico de Datos

3.1. Métricas Cuantitativas y Cualitativas

Cuadro 1: Métricas de evaluación

Métrica	Definición	Tipo
Tiempo promedio de corrección de errores de borrowing	Minutos empleados en resolver cada error de compilación relacionado con préstamos.	Cuantitativa
Tasa de éxito en proyecto final sin IA	Porcentaje de pruebas unitarias superadas en el proyecto final.	Cuantitativa
Número de invocaciones a la IA (solo JIT)	Conteo de consultas a asistentes durante las semanas 3–4.	Cuantitativa
Comprensión teórica de la memoria	Puntaje en examen conceptual (0–100).	Cuantitativa
Capacidad de refactorización	Evaluación cualitativa por pares ciegos de la claridad y eficiencia del código final.	Cualitativa (escala Likert)

3.2. Instrumentos

- **Registros automáticos:** Logs del compilador y de uso de IA (para el grupo JIT).
- **Cuestionarios:** Examen teórico diseñado por expertos en lenguajes.

- **Rúbrica de evaluación de código:** Criterios de legibilidad, uso correcto de ownership, manejo de errores.

3.3. Análisis Estadístico

- **Contraste de hipótesis:** Se utilizará la prueba t de Student para muestras independientes (si los datos son normales) o la U de Mann-Whitney (si no). Nivel de significancia $\alpha = 0,05$.
- **Hipótesis nula (H_0):** No hay diferencias significativas en el rendimiento entre ambos grupos.
- **Hipótesis alternativa (H_1):** El grupo AaA supera al JIT en al menos dos de las métricas principales (tiempo de corrección, examen teórico y proyecto final).
- **Análisis de correlación:** Para el grupo JIT, se explorará la relación entre el número de invocaciones a la IA y el rendimiento final, usando coeficiente de Pearson.
- **Análisis cualitativo:** Las evaluaciones de refactorización se analizarán mediante comparación de medianas (prueba de Wilcoxon).

4. Conclusiones Preliminares y Discusión Pedagógica

4.1. Resultados esperados

Si H_0 se rechaza, se confirmaría que la instrucción teórica previa (AaA) proporciona una base más sólida para enfrentar problemas complejos de memoria, mientras que el enfoque JIT podría generar una dependencia de la IA que limite la comprensión profunda. No obstante, es posible que el grupo JIT muestre mayor agilidad en tareas rutinarias, gracias al entrenamiento práctico intensivo.

4.2. Implicaciones para la teoría de Raj Reddy

Reddy sostiene que los tutores virtuales permiten un aprendizaje autodirigido y eficiente. Sin embargo, este estudio podría matizar su postura: la IA es una herramienta poderosa para resolver problemas concretos, pero no necesariamente fomenta la construcción de modelos mentales abstractos. Si el grupo AaA supera al JIT en comprensión teórica y refactorización, se sugeriría que el *aprender a aprender* debe preceder o acompañar el uso de asistentes, para evitar una ilusión de competencia".

Por otro lado, si el grupo JIT iguala o supera al AaA en el proyecto final, se reforzaría la idea de que el aprendizaje justo a tiempo, apoyado por IA, es viable y puede democratizar el acceso a lenguajes complejos. En cualquier caso, los resultados orientarán el diseño de futuros cursos de programación, equilibrando la instrucción teórica con el uso estratégico de herramientas inteligentes.

Referencias

- [1] Reddy, R. (2023). *Autodidaxia in the Age of AI: The Role of Virtual Tutors*. (Referencia hipotética; se recomienda citar la obra original).
- [2] The Rust Programming Language, *The Rust Book*. (2024).
- [3] S. Klusener et al., “Learning Rust: A Study of Novice Difficulties,” *ACM SIGCSE*, 2023.

```
# Ejemplo de tipos numericos
a = 10 # Entero
b = 3.14 # Punto flotante
c = a + b
print(c) # Salida: 13.14
```

13.14

```
# Ejemplo de cadena de texto
nombre = "Juan"
saludo = "Hola," + nombre
print(saludo) # Salida: Hola, Juan
```

Hola, Juan

```
print(f"juan recibe despues de decir {saludo} un vuelto de {c}")
```

juan recibe despues de decir Hola, Juan un vuelto de 13.14

```
import math
print(math.pi)
```

3.141592653589793

```
import math
print(f"{math.pi:.10f}")
```

3.1415926536

```
print("lea con cuidado lo que se pide")
a=input("ingrese el numero entero deseado de cifras significativas")
import math
print(f"El numero pi con {a} cifras decimales es igual a: {math.pi:.{a}f}")
```

lea con cuidado lo que se pide
ingrese el numero entero deseado de cifras significativas5
El numero pi con 5 cifras decimales es igual a: 3.14159

```
# Python como calculadora
print("=== Operaciones Basicas ===")
print(f"Suma: 2 + 3 = {2 + 3}")
print(f"Resta: 10 - 4 = {10 - 4}")
print(f"Multiplicacion: 6 * 7 = {6 * 7}")
print(f"Division: 15 / 3 = {15 / 3}")
print(f"Potencia: 2 ** 8 = {2 ** 8}")
print(f"Modulo (Residuo): 17 % 5 = {17 % 5}")
```

```
=== Operaciones Basicas ===
Suma: 2 + 3 = 5
Resta: 10 - 4 = 6
Multiplicacion: 6 * 7 = 42
Division: 15 / 3 = 5.0
Potencia: 2 ** 8 = 256
Modulo (Residuo): 17 % 5 = 2
```

```
def brahian_suma(a,b):
    return f"esta es la suma de {a} y {b} cuyo resultado es: {a+b}"
```

```
brahian_suma (2,1)
```

'esta es la suma de 2 y 1 cuyo resultado es: 3'

```
def brahian_rest(a,b):
    return f"esta es la resta de {a} y {b} cuyo resultado es: {a-b}"
```

```
brahian_rest (2,1)
```

'esta es la resta de 2 y 1 cuyo resultado es: 1'

```
def Multiplicacion(a,b):
    return f"esta es la multiplicacion de {a} y {b} cuyo resultado es: {a*b}"
```

```
Multiplicacion (6,2)
```

'esta es la multiplicacion de 6 y 2 cuyo resultado es: 12'

```
def divicion(a,b):  
    return f"esta es la divicion de {a} y {b} cuyo resultado es: {a/b}"  
  
divicion (7,1)
```

```
'esta es la divicion de 7 y 1 cuyo resultado es: 7.0'
```

```
def potenciacion(a,b):  
    return f"esta es la potenciacion de {a} y {b} cuyo resultado es: {a**b}"  
  
potenciacion (3,2)
```

```
'esta es la potenciacion de 3 y 2 cuyo resultado es: 9'
```

```
def Residuo(a,b):  
    return f"esta es el residuo de {a} y {b} cuyo resultado es: {a+b}"  
  
Residuo (8,3)
```

```
'esta es el residuo de 8 y 3 cuyo resultado es: 11'
```

```
# funcion para hallar el area de un cuadrado, conociendo su lado  
def area_cuadrado(lado):  
    return f"el area del cuadrado de lado {lado} es: {lado**2} unidades cuadradas"  
  
area_cuadrado(5)
```

```
'el area del cuadrado de lado 5 es: 25 unidades cuadradas'
```

```
# funcion para hallar el area de un rectangulo, conociendo su altura y su base  
def area_rectanculo(base,altura):  
    return f"el area del resctangulo de base {base} y altura {altura} es: {base*altura} unidades cuadradas"  
  
area_rectanculo(5)
```

```
-----  
TypeError                                Traceback (most recent call last)  
/tmp/ipykernel_722/2251255018.py in <cell line: 0>()  
      3     return f"el area del resctangulo de base {base} y altura {altura} es: {base*altura} unidades cuadradas"  
      4  
----> 5 area_rectanculo(5)  
  
TypeError: area_rectanculo() missing 1 required positional argument: 'altura'
```



```
def brahian_raiz(x):  
    y= (3+x**(1/2))/((x-1)**(1/2)+3)  
    return y  
brahian_raiz(1)
```

```
1.3333333333333333
```

```
def brahian_raiz(x,n):  
    y= (3+x**(1/2))/((x-1)**(1/2)+3)  
    w= f"{y:.{n}f}"  
    return w  
  
brahian_raiz(1,14)
```

```
'1.3333333333333333'
```

```
import math
```

Regiones Geométricas y sus Áreas

Guía Completa para el Cálculo de Áreas

Brahian Marín Giraldo

Biología 2026-2

Universidad CES

6 de agosto de 2026

Resumen

Este documento presenta un compendio exhaustivo de las principales regiones geométricas planas y sus respectivas fórmulas para el cálculo del área. Se incluyen figuras básicas (triángulos, cuadriláteros, círculos), figuras compuestas, regiones bajo curvas, y áreas en coordenadas polares. Cada sección contiene la definición de la región, su fórmula matemática y ejemplos prácticos. El objetivo es proporcionar una referencia completa para estudiantes y profesionales que necesiten calcular áreas en diversos contextos matemáticos y de ingeniería.

Índice

1. Introducción	3
2. Regiones Poligonales Básicas	3
2.1. Triángulos	3
2.1.1. Fórmulas para el área de triángulos	3
2.1.2. Ejemplo	3
2.2. Cuadriláteros	3
2.2.1. Rectángulo	3
2.2.2. Cuadrado	3
2.2.3. Paralelogramo	3
2.2.4. Rombo	4
2.2.5. Trapecio	4
2.2.6. Ejemplo	4
3. Regiones Circulares	4
3.1. Círculo	4
3.2. Sector Circular	4
3.3. Segmento Circular	4
3.4. Corona Circular	4
4. Regiones en el Plano Cartesiano	4
4.1. Área bajo una curva	4
4.2. Área entre dos curvas	5
4.3. Área en coordenadas polares	5
5. Regiones Especiales	5
5.1. Elipse	5
5.2. Parábola y región parabólica	5
5.3. Región entre una curva y el eje y	5

6. Regiones Compuestas	5
6.1. Estrategia para calcular áreas compuestas	6
6.2. Ejemplo de región compuesta	6
7. Regiones en el Espacio (Áreas de Superficies)	6
7.1. Cilindro (área lateral y total)	6
7.2. Cono	6
7.3. Esfera	6
8. Tabla Resumen de Áreas	6
9. Aplicaciones Prácticas	8
9.1. Topografía y medición de terrenos	8
9.2. Ingeniería civil y arquitectura	8
9.3. Biología y ecología	8
9.4. Física y mecánica	8
9.5. Estadística y probabilidad	8
10. Conclusiones	8

1. Introducción

El cálculo de áreas es fundamental en matemáticas, física, ingeniería y ciencias aplicadas. Una **región** es un conjunto de puntos en el plano que forma una figura con límites definidos. Conocer las fórmulas para calcular el área de diferentes regiones permite resolver problemas prácticos como:

- Diseño de estructuras y superficies.
- Cálculo de terrenos y propiedades.
- Optimización de materiales y recursos.
- Modelado de fenómenos naturales.
- Análisis de datos y estadística.

Este documento cubre desde las regiones más elementales hasta las más complejas, incluyendo tanto figuras regulares como irregulares.

2. Regiones Poligonales Básicas

2.1. Triángulos

Un triángulo es una región delimitada por tres segmentos de recta que se intersectan en tres puntos no colineales.

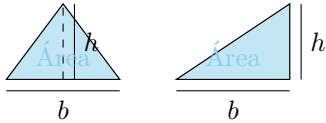


Figura 1: Tipos de triángulos

2.1.1. Fórmulas para el área de triángulos

1. Área con base y altura:

$$A = \frac{1}{2} \cdot b \cdot h \quad (1)$$

donde b es la base y h es la altura perpendicular a esa base.

2. Fórmula de Herón (con tres lados):

$$A = \sqrt{s(s-a)(s-b)(s-c)} \quad (2)$$

donde a, b, c son los lados del triángulo y $s = \frac{a+b+c}{2}$ es el semiperímetro.

3. Área con dos lados y el ángulo entre ellos:

$$A = \frac{1}{2} \cdot a \cdot b \cdot \sin(\theta) \quad (3)$$

donde a y b son dos lados y θ es el ángulo entre ellos.

4. Área de un triángulo rectángulo:

$$A = \frac{1}{2} \cdot \text{cateto}_1 \cdot \text{cateto}_2 \quad (4)$$

2.1.2. Ejemplo

Calcular el área de un triángulo con base 8 cm y altura 6 cm.

$$A = \frac{1}{2} \cdot 8 \cdot 6 = 24 \text{ cm}^2$$

2.2. Cuadriláteros

Los cuadriláteros son regiones con cuatro lados. Incluyen rectángulos, cuadrados, paralelogramos, rombos y trapecios.

2.2.1. Rectángulo

$$A = b \cdot h$$

donde b es la base y h es la altura.

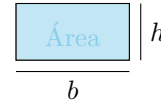


Figura 2: Rectángulo

2.2.2. Cuadrado

$$A = l^2$$

donde l es la longitud del lado.

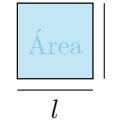


Figura 3: Cuadrado

2.2.3. Paralelogramo

$$A = b \cdot h$$

donde b es la base y h es la altura perpendicular.

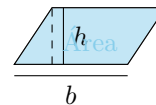


Figura 4: Paralelogramo

2.2.4. Rombo

$$A = \frac{d_1 \cdot d_2}{2}$$

donde d_1 y d_2 son las diagonales.

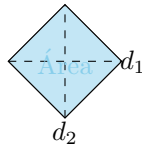


Figura 5: Rombo

2.2.5. Trapecio

$$A = \frac{(B + b) \cdot h}{2}$$

donde B es la base mayor, b es la base menor y h es la altura.

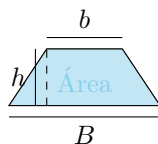


Figura 6: Trapecio

2.2.6. Ejemplo

Calcular el área de un trapecio con bases 10 cm y 6 cm, y altura 4 cm.

$$A = \frac{(10 + 6) \cdot 4}{2} = \frac{16 \cdot 4}{2} = 32 \text{ cm}^2$$

3. Regiones Circulares

3.1. Círculo

El círculo es una región delimitada por una circunferencia.

$$A = \pi \cdot r^2$$

donde r es el radio del círculo.

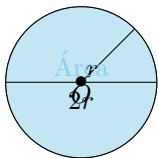


Figura 7: Círculo con radio r

3.2. Sector Circular

Un sector circular es una porción del círculo delimitada por dos radios y el arco entre ellos.

$$A = \frac{\theta}{360^\circ} \cdot \pi \cdot r^2 = \frac{1}{2} \cdot r^2 \cdot \theta$$

donde θ es el ángulo del sector en grados o radianes, y r es el radio.



Figura 8: Sector circular

3.3. Segmento Circular

Un segmento circular es la región entre una cuerda y el arco que la subtiende.

$$A = \frac{r^2}{2} \cdot (\theta - \sin \theta)$$

donde θ es el ángulo central en radianes.

3.4. Corona Circular

La corona circular es la región entre dos círculos concéntricos.

$$A = \pi \cdot (R^2 - r^2)$$

donde R es el radio mayor y r es el radio menor.

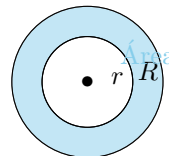


Figura 9: Corona circular

4. Regiones en el Plano Cartesiano

4.1. Área bajo una curva

El área bajo una curva $y = f(x)$ entre $x = a$ y $x = b$ se calcula mediante la integral definida:

$$A = \int_a^b f(x) dx$$

siempre que $f(x) \geq 0$ en $[a, b]$.

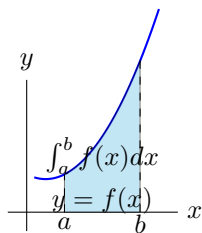


Figura 10: Área bajo la curva $y = f(x)$

4.2. Área entre dos curvas

El área entre dos curvas $y = f(x)$ y $y = g(x)$ entre $x = a$ y $x = b$, donde $f(x) \geq g(x)$, es:

$$A = \int_a^b [f(x) - g(x)] dx$$

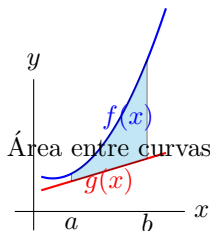


Figura 11: Área entre dos curvas

4.3. Área en coordenadas polares

Para una curva en coordenadas polares $r = f(\theta)$, el área barrida entre $\theta = \alpha$ y $\theta = \beta$ es:

$$A = \frac{1}{2} \int_{\alpha}^{\beta} [f(\theta)]^2 d\theta$$

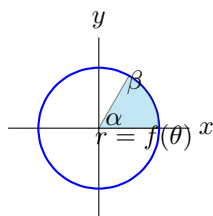


Figura 12: Área en coordenadas polares

5. Regiones Especiales

5.1. Elipse

La elipse es una región definida por la ecuación $\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$.

$$A = \pi \cdot a \cdot b$$

donde a es el semieje mayor y b es el semieje menor.

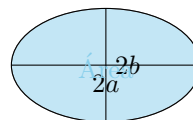


Figura 13: Elipse

5.2. Parábola y región parabólica

El área bajo una parábola de la forma $y = ax^2 + bx + c$ desde $x = x_1$ hasta $x = x_2$ se calcula con la integral definida.

Para una parábola específica $y = ax^2$, el área entre $x = -r$ y $x = r$ es:

$$A = \frac{2}{3} \cdot \text{base} \cdot \text{altura} = \frac{2}{3} \cdot 2r \cdot ar^2 = \frac{4}{3} ar^3$$

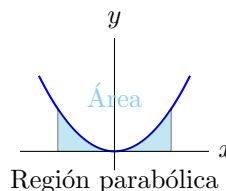


Figura 14: Región parabólica

5.3. Región entre una curva y el eje y

El área entre una curva $x = f(y)$ y el eje y entre $y = c$ y $y = d$ es:

$$A = \int_c^d f(y) dy$$

6. Regiones Compuestas

Las regiones compuestas se forman por la unión de varias figuras básicas. El área total se calcula sumando o restando áreas de figuras individuales.

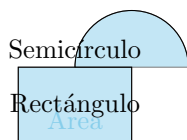


Figura 15: Región compuesta: rectángulo con semicírculo

7.3. Esfera

$$A = 4\pi \cdot r^2$$

8. Tabla Resumen de Áreas

6.1. Estrategia para calcular áreas compuestas

1. Identificar las figuras geométricas que componen la región.
2. Calcular el área de cada figura individual.
3. Sumar las áreas si se trata de una unión.
4. Restar áreas si hay superposiciones o huecos.
5. Asegurarse de que todas las medidas estén en las mismas unidades.

6.2. Ejemplo de región compuesta

Calcular el área de la siguiente figura: un rectángulo de 10 cm de ancho y 6 cm de alto, con un semicírculo de radio 3 cm en la parte superior.

- Área del rectángulo: $A_r = 10 \cdot 6 = 60 \text{ cm}^2$
- Área del semicírculo: $A_s = \frac{1}{2} \cdot \pi \cdot 3^2 = \frac{9\pi}{2} \approx 14,14 \text{ cm}^2$
- Área total: $A_t = 60 + 14,14 = 74,14 \text{ cm}^2$

7. Regiones en el Espacio (Áreas de Superficies)

Aunque el enfoque principal de este documento es el área de regiones planas, también es importante mencionar algunas superficies tridimensionales:

7.1. Cilindro (área lateral y total)

- Área lateral: $A_L = 2\pi \cdot r \cdot h$
- Área total: $A_T = 2\pi \cdot r \cdot h + 2\pi \cdot r^2 = 2\pi r(r + h)$

7.2. Cono

- Área lateral: $A_L = \pi \cdot r \cdot g$ (donde g es la generatriz)
- Área total: $A_T = \pi \cdot r \cdot g + \pi \cdot r^2$

Cuadro 1: Resumen de fórmulas de áreas

Región	Fórmula del área
Triángulo	$A = \frac{1}{2} \cdot b \cdot h$
Rectángulo	$A = b \cdot h$
Cuadrado	$A = l^2$
Paralelogramo	$A = b \cdot h$
Rombo	$A = \frac{d_1 \cdot d_2}{2}$
Trapezio	$A = \frac{(B+b) \cdot h}{2}$
Círculo	$A = \pi \cdot r^2$
Sector circular	$A = \frac{\theta}{2} \cdot r^2$
Corona circular	$A = \pi \cdot (R^2 - r^2)$
Elipse	$A = \pi \cdot a \cdot b$
Superficie esférica	$A = 4\pi \cdot r^2$
Cilindro (lateral)	$A = 2\pi \cdot r \cdot h$
Cono (lateral)	$A = \pi \cdot r \cdot g$

9. Aplicaciones Prácticas

9.1. Topografía y medición de terrenos

El cálculo de áreas es fundamental para medir extensiones de terreno, parcelas y propiedades.

9.2. Ingeniería civil y arquitectura

Se utiliza para calcular superficies de construcción, materiales necesarios y costos.

9.3. Biología y ecología

Permite estimar áreas de hábitats, superficies de cultivo y poblaciones.

9.4. Física y mecánica

Esencial para calcular momentos de inercia, centros de masa y fuerzas.

9.5. Estadística y probabilidad

Se usa para calcular áreas bajo curvas de distribución de probabilidad.

10. Conclusiones

El conocimiento de las diferentes regiones geométricas y sus fórmulas de área es fundamental en múltiples disciplinas. Desde figuras básicas hasta curvas complejas, cada región tiene su propio método de cálculo. La comprensión de estos conceptos permite:

- Resolver problemas prácticos de manera eficiente.
- Modelar fenómenos naturales y sociales.
- Optimizar recursos y diseños.
- Desarrollar pensamiento espacial y matemático.

La práctica constante y la aplicación de estas fórmulas en problemas reales consolidan el aprendizaje y preparan para situaciones más complejas en el ámbito profesional.

Referencias

- [1] Stewart, J. (2015). *Cálculo: Trascendentes Tempranas*. Cengage Learning.
- [2] Moise, E. E. (1990). *Geometría Moderna*. Addison-Wesley.
- [3] Spiegel, M. R. (1998). *Manual de Fórmulas Matemáticas*. McGraw-Hill.
- [4] Apostol, T. M. (1967). *Cálculus*. Editorial Reverté.